Fondements informatiques I

Cours 9: Algorithmes : piles et files

Sorina Ionica sorina.ionica@uvsq.fr

Bilal FAYE bilal.faye@uvsq.fr

File : définition et principe

Une **file** (en anglais, *queue*) est une structure de données dans laquelle :

- les éléments sont ajoutés à la fin de la file ;
- les éléments sont retirés au début de la file.

Exemple concret

Comme dans une file d'attente à la caisse : la première personne arrivée est la première à être servie.

File : définition et principe

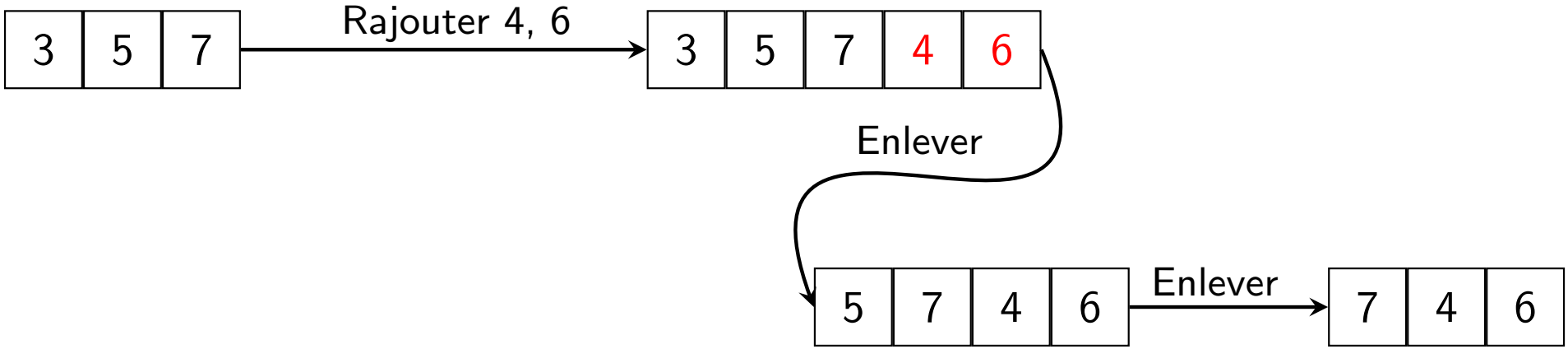
Une **file** (en anglais, *queue*) est une structure de données dans laquelle :

- les éléments sont ajoutés à la fin de la file ;
- les éléments sont retirés au début de la file.

Exemple concret

Comme dans une file d'attente à la caisse : la première personne arrivée est la première à être servie.

FIFO : First In, First Out



File : définition et principe

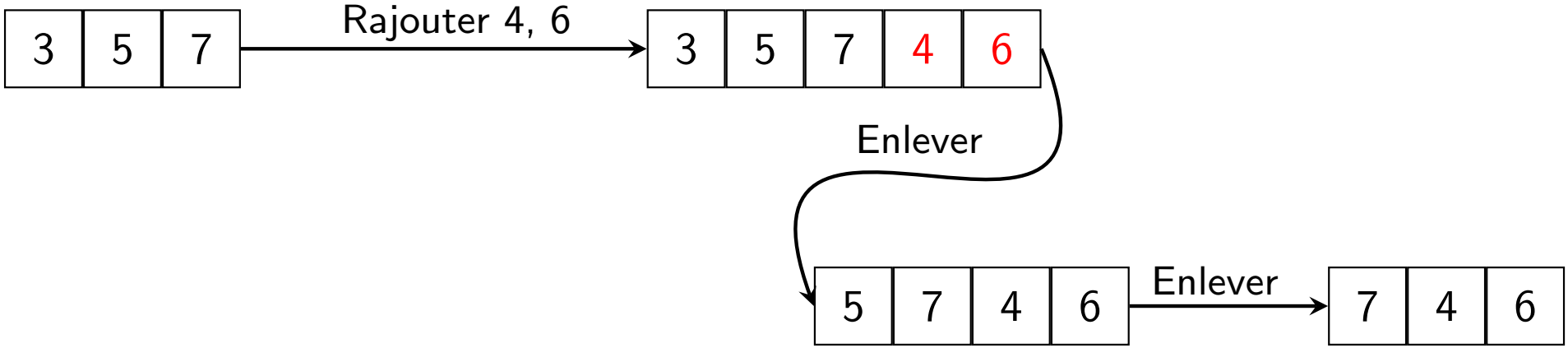
Une **file** (en anglais, *queue*) est une structure de données dans laquelle :

- les éléments sont ajoutés à la fin de la file ;
- les éléments sont retirés au début de la file.

Exemple concret

Comme dans une file d'attente à la caisse : la première personne arrivée est la première à être servie.

FIFO : First In, First Out



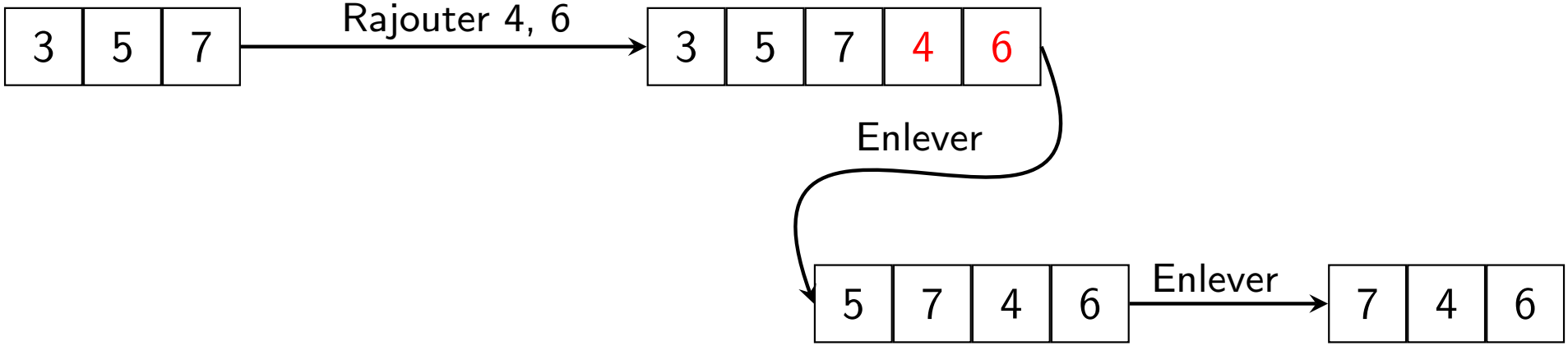
Applications :

- traitement des tâches dans leur ordre d'arrivée ;
- gestion des demandes d'impression ;
- transmission de données dans un réseau.

File : les opérations principales

Pour manipuler une file, on utilise deux opérations :

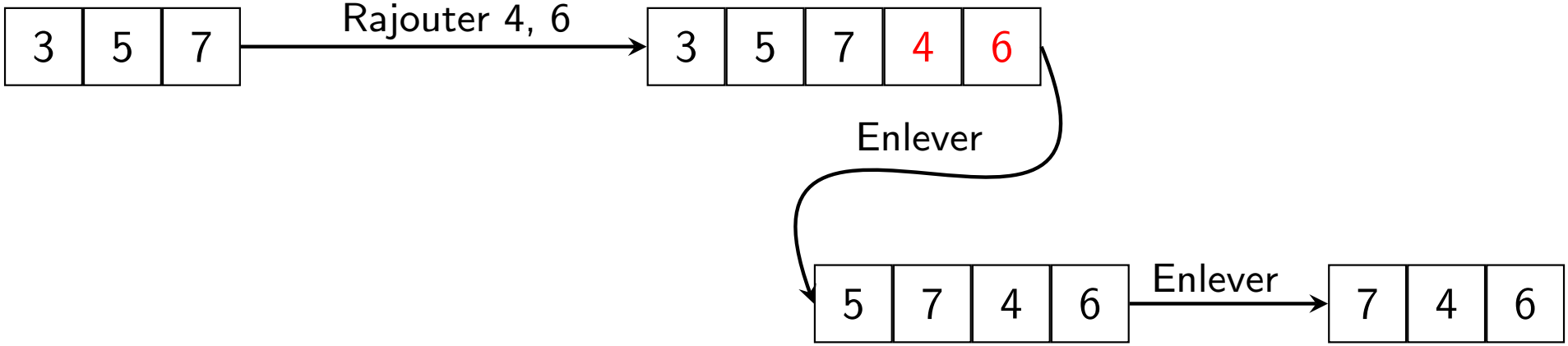
- **Ajouter :** insérer un élément à la fin de la file.
- **Enlever :** retirer l'élément situé au début de la file.



File : les opérations principales

Pour manipuler une file, on utilise deux opérations :

- **Ajouter :** insérer un élément à la fin de la file.
- **Enlever :** retirer l'élément situé au début de la file.



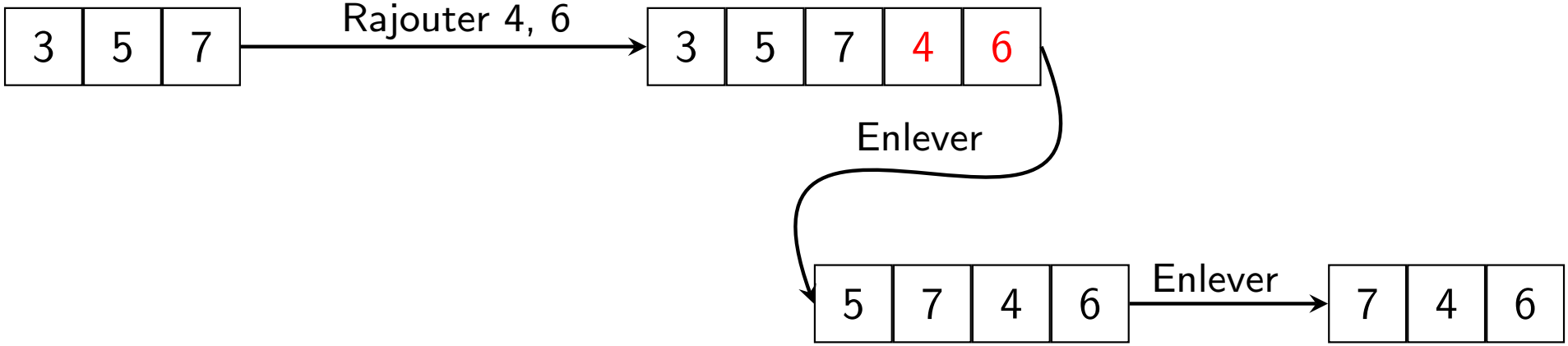
Exemple

```
file = [3, 5, 7]
ajouter(file, 4)
ajouter(file, 6)
enlever(file)
enlever(file)
```

File : les opérations principales

Pour manipuler une file, on utilise deux opérations :

- **Ajouter :** insérer un élément à la fin de la file.
- **Enlever :** retirer l'élément situé au début de la file.



Exemple

```
file = [3, 5, 7]
ajouter(file, 4)
ajouter(file, 6)
enlever(file)
enlever(file)
```

Les éléments sont retirés dans le même ordre que celui dans lequel ils ont été ajoutés.

Ajouter dans une file

On suppose que la file est représentée par un tableau dynamique.

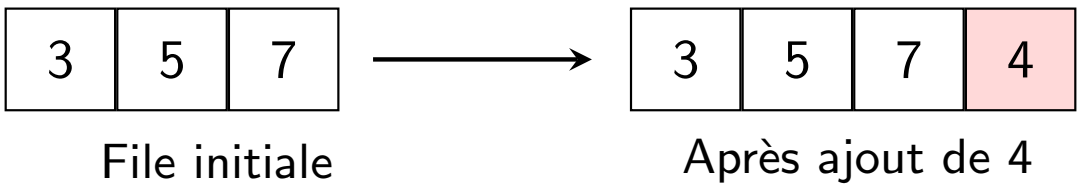
Principe

Un nouvel élément est toujours ajouté à la **fin de la file**.

Algorithme : ajouter(f,e)

```
Entrée : une file f et un élément e
Sortie : la file f modifiée

Ajouter e à la fin de f
```



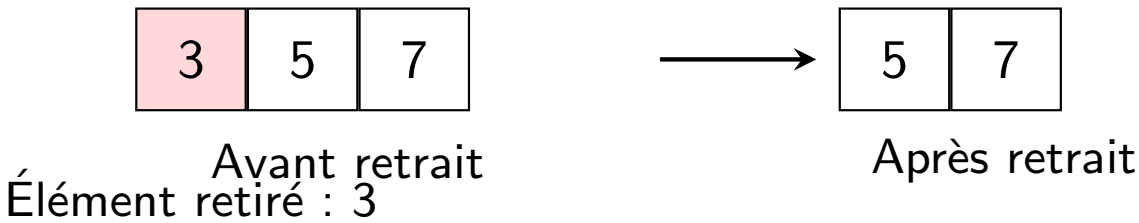
Enlever dans une file

On retire toujours l'élément situé au **début de la file**.

Principe : L'élément retiré est le premier élément de la file. Les autres éléments avancent d'une position.

Algorithme : enlever(f)

```
Entrée : une file f
Sortie : l'élément retiré et la file modifiée
Si f est vide :
    Signaler que la file est vide
Sinon :
    e <- premier élément de f
    Supprimer le premier élément de f
Retourner e
```



Une file en Python

En Python, on peut utiliser une liste pour représenter une file.

Ajouter un élément

```
def ajouter(f, e):
    f.append(e)
```

L'élément est ajouté à la fin de la liste.

Enlever un élément

```
def enlever(f):
    if len(f) > 0:
        return f.pop(0)
    return None
```

Le premier élément est retiré et renvoyé.

Exemple d'utilisation

```
f = [3, 5, 7]
ajouter(f, 4)
print(f)            # [3, 5, 7, 4]
e = enlever(f)
print(e)            # 3
print(f)            # [5, 7, 4]
```

Suivre les opérations sur une file

On considère la file initiale :

```
f = [3, 5, 7]
```

Instructions Python

```
ajouter(f, 4)
ajouter(f, 6)
e1 = enlever(f)
e2 = enlever(f)
print(f)
print(e1, e2)
```

Suivre les opérations sur une file

On considère la file initiale :

```
f = [3, 5, 7]
```

Instructions Python

```
ajouter(f, 4)
ajouter(f, 6)
e1 = enlever(f)
e2 = enlever(f)
print(f)
print(e1, e2)
```

Résultat

- File finale : `[7, 4, 6]`
- Premier élément retiré : 3
- Deuxième élément retiré : 5

Les premiers éléments ajoutés sont les premiers retirés.

Exemple d'utilisation : gestion des impressions

Une imprimante reçoit plusieurs demandes d'impression.

Situation initiale

Trois documents sont en attente : $f = [\texttt{Doc1}, \texttt{Doc2}, \texttt{Doc3}]$ Un nouveau document Doc4 arrive.

Programme Python

```
f = ["Doc1", "Doc2", "Doc3"]
ajouter(f, "Doc4")
while len(f) > 0:
    document = enlever(f)
    print("Impression :", document)
```

Exemple d'utilisation : gestion des impressions

Une imprimante reçoit plusieurs demandes d'impression.

Situation initiale

Trois documents sont en attente : $f = [\texttt{Doc1}, \texttt{Doc2}, \texttt{Doc3}]$ Un nouveau document Doc4 arrive.

Programme Python

```
f = ["Doc1", "Doc2", "Doc3"]
ajouter(f, "Doc4")
while len(f) > 0:
    document = enlever(f)
    print("Impression :", document)
```

Résultat

```
Impression : Doc1
Impression : Doc2
Impression : Doc3
Impression : Doc4
```

Pile : définition et principe

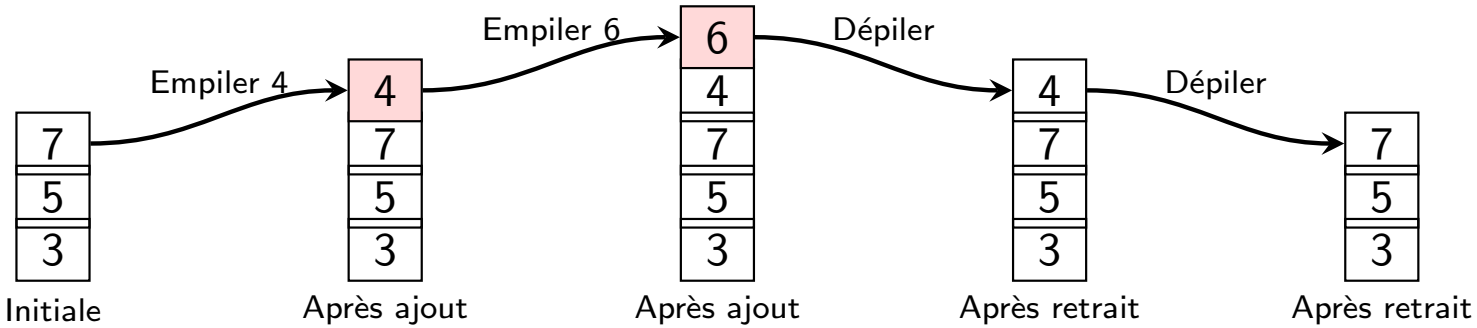
Une **pile** (en anglais, *stack*) est une structure de données dans laquelle :

- les éléments sont ajoutés au sommet de la pile ;
- les éléments sont retirés du sommet de la pile.

Exemple concret

Comme une pile d'assiettes : on pose une assiette au-dessus des autres et on retire toujours celle du dessus.

LIFO : Last In, First Out



Applications :

- inversion d'une séquence ;
- évaluation d'expressions arithmétiques ;
- historique de navigation dans un navigateur.

Pile : représentation et empilement

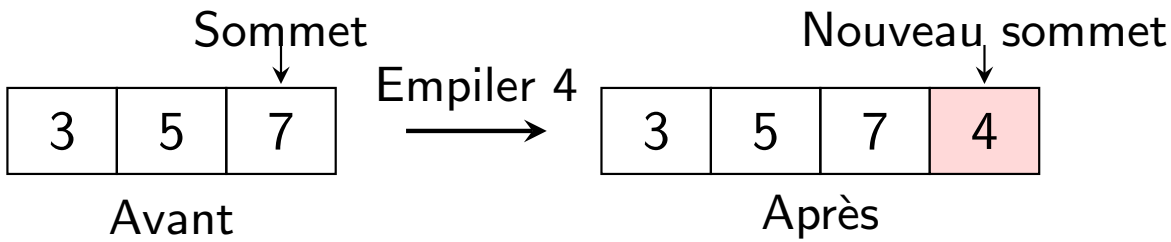
On représente la pile par un tableau dynamique.

- Le sommet correspond à la dernière case du tableau.
- Empiler consiste à ajouter un élément à la fin.

Algorithme : empiler(p,e)

```
Entree : une pile p et un element e
Sortie : la pile p modifiee

Ajouter e a la fin de p
```



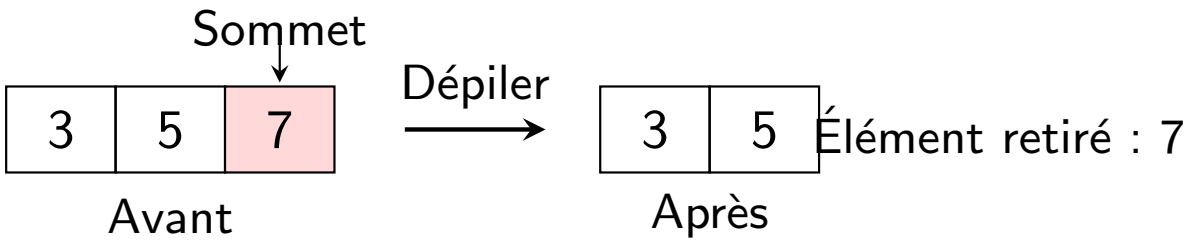
Pile : dépiler

Dépiler consiste à retirer l'élément situé au sommet de la pile.

Algorithme : depiler(p)

```
Entree : une pile p
Sortie : l'element retire

Si p est vide :
    Signaler que la pile est vide
Sinon :
    e <- dernier element de p
    Supprimer le dernier element de p
    Retourner e
```



Une pile en Python

En Python, on utilise une liste pour représenter une pile.

Empiler

```
def empiler(p, e):
    p.append(e)
```

Ajoute l'element au sommet.

Dépiler

```
def depiler(p):
    if len(p) > 0:
        return p.pop()
    return None
```

Retire et renvoie le sommet.

Exemple d'utilisation

```
p = [3, 5, 7]
empiler(p, 4)
empiler(p, 6)
print(p)            # [3, 5, 7, 4, 6]
depiler(p)
depiler(p)
print(p)            # [3, 5, 7]
```

Exemple d'utilisation : inverser une séquence

Une pile permet d'inverser l'ordre des éléments d'un tableau.

Principe

On retire successivement les éléments du tableau et on les empile dans une nouvelle pile.

Programme Python

```
a = [3, 5, 7, 2]
inverse = []
while len(a) > 0:
    e = depiler(a)
    empiler(inverse, e)
print(inverse)
```

Exemple d'utilisation : inverser une séquence

Une pile permet d'inverser l'ordre des éléments d'un tableau.

Principe

On retire successivement les éléments du tableau et on les empile dans une nouvelle pile.

Programme Python

```
a = [3, 5, 7, 2]
inverse = []
while len(a) > 0:
    e = depiler(a)
    empiler(inverse, e)
print(inverse)
```

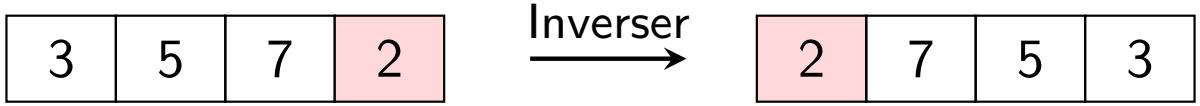


Tableau initial — Séquence inversée